

SmartB2B Implementation Plan

Strategic 1.5-Hour Sprint Modules for Enterprise B2B Architecture

This report breaks down the development of the SmartB2B enterprise-grade SaaS platform into focused, 1.5-hour engineering sprints. By approaching the system modularly, we address complex distributed system challenges—such as race conditions, asynchronous processing, and strict financial ledgers—systematically and securely.

Module 1: Foundation & Replica Set Configuration

The Sub-Problem We Are Solving:

Before writing business logic, the system needs a rock-solid data foundation. Regular MongoDB won't work here; we must set up a Replica Set to enable multi-document ACID transactions, which are mandatory for financial ledger integrity.

Prerequisites:

Node.js installed, local or cloud MongoDB, basic Express.js knowledge.

Steps to Build:

- Initialize a Node.js (Express.js or NestJS) project.
- Configure a MongoDB Replica Set locally (using Docker or MongoDB Atlas).
- Create the Mongoose schemas exactly as specified in the blueprint: Users & Roles, Products, Retailer Credit Profile, and Ledger Transactions.

Module 2: Role-Based Access Control (RBAC) & Tenant Isolation

The Sub-Problem We Are Solving:

The platform must support two discrete interfaces (Wholesaler vs. Retailer). We need to ensure that a retailer cannot access wholesale configuration panels.

Prerequisites:

Module 1 complete, understanding of JWT (JSON Web Tokens) and Express middleware.

Steps to Build:

- Build a registration and login route for the Users & Roles collection.
- Generate a JWT upon login that includes the user's ID and role (WHOLESALE or RETAILER).
- Write an authorization middleware function that strictly isolates routes so retailers can only access the catalog, and wholesalers can access ledger analytics.

Module 3: Catalog Management & High-Speed Caching

The Sub-Problem We Are Solving:

Wholesalers handle thousands of line items. If every retailer fetches the product list directly from MongoDB, the primary database will crash under stress.

Prerequisites:

Module 1 & 2, Redis installed locally or via cloud.

Steps to Build:

- Create a POST route for wholesalers to add items to the Products collection.
- Create a GET route for retailers to browse the digital product catalog.
- Implement Redis caching on the GET route: check Redis first; if data is missing, query MongoDB, return the data to the user, and save it in Redis for the next request.

Module 4: The ACID-Compliant Checkout (The "Double Spend" Fix)

The Sub-Problem We Are Solving:

If a retailer submits two orders simultaneously in different tabs, a naive backend would check their credit balance twice and allow both, exceeding their limit.

Prerequisites:

Module 3, solid understanding of MongoDB Session Transactions.

Steps to Build:

- Create the order submission route.
- Implement `session.startTransaction()` to lock the data.
- Verify the core business rule: check if the new order value plus the `outstandingBalance` surpasses the `creditLimit`. Block the order immediately if it does.
- If approved, write the order and update the Retailer Credit Profile. Commit the transaction, or abort it if any step fails.

Module 5: Inventory Race Conditions (The Over-selling Fix)

The Sub-Problem We Are Solving:

High-concurrency checkouts can cause stock numbers to drop below zero if two retailers buy the same item at the exact same millisecond.

Prerequisites:

Module 4 code.

Steps to Build:

- Modify the transaction block from Module 4.
- When updating the Products collection, do not just calculate the new stock in Node.js.
- Instead, implement atomicity using the `$inc` operator (`$inc: { stockQuantity: -orderedQty }`).
- Combine this with a query filter ensuring `$gte: orderedQty` so the database rejects the update if stock is insufficient.

Module 6: Real-Time Event Pipelines

The Sub-Problem We Are Solving:

When a retailer checks out, the wholesaler needs an immediate alert without refreshing their dashboard.

Prerequisites:

Basic understanding of WebSockets/Socket.io.

Steps to Build:

- Attach Socket.io to your Express server.
- On the frontend (React), establish a connection to the Socket server upon login.
- Inside your Module 4/5 checkout route, right after a successful transaction commit, emit an event (e.g., `new_order_received`) containing the order details.
- Update the wholesaler's React dashboard to listen for this event and inject the new order into the live fulfillment pipeline dashboard.

Module 7: Resilient Background Workers (PDF Invoices)

The Sub-Problem We Are Solving:

Generating a multi-page PDF invoice and pushing it to AWS S3 takes up to 2 seconds, which blocks the Node.js event loop and freezes the application.

Prerequisites:

AWS S3 account/bucket, Redis running, BullMQ installed.

Steps to Build:

- Configure a BullMQ/Redis Stream queue on your server.
- Instead of generating the PDF in the checkout route, push a job containing the `orderId` to the queue.
- Create a separate asynchronous worker script that listens to this queue, generates the PDF, and securely uploads it to AWS S3 for immutable remote cloud hosting.